计算机图形学实验报告：实验三Poisson Image Editing

姓名：宁尚哲　学号：PB24000099

March 22, 2026

Abstract

本实验基于Framework2D 框架，实现了经典的Poisson Image Editing 算法。通过将图像克隆问题转化为离散泊松方程的求解，实现了无缝的图像融合（Seamless Cloning）。实验中利用Eigen 库构建并求解大规模稀疏线性方程组，并针对交互体验实现了矩阵预分解的实时更新策略。此外，实验扩展支持了Mixing Gradients 模式，以及基于扫描线算法的复杂不规则区域选择。实验结果证明，该方法能有效消除图像克隆时的边界断裂感，使融合结果在视觉上更加自然。

1　引言

在图像编辑中，传统的“剪切-粘贴”操作（Paste）往往会在目标图像中留下明显的边界接缝，这是由于源图像与目标图像背景颜色不一致导致的。Poisson Image Editing 提出在梯度域进行编辑：保持源区域内部的梯度信息（细节），同时通过求解偏微分方程使区域边界平滑过渡到目标背景颜色。

2　算法原理

2.1　泊松方程与变分模型

图像融合问题可建模为在已知边界条件下的泛函极小化问题。对于目标区域Ω，我们希望寻找一个函数f，使得其梯度∇f 尽可能接近引导矢量场$\mathbf{v}$（通常取自源图像∇g）：

$$\min_f \iint_\Omega |\nabla f - \mathbf{v}|^2 \, dA \quad \text{with} \quad f|_{\partial\Omega} = f^*|_{\partial\Omega} \tag{1}$$

其中f^* 是目标图像，$\mathbf{v}$ 是引导矢量场，$\partial\Omega$ 是区域边界。这个能量泛函的含义是：在满足边界条件的前提下，使融合图像的梯度场尽可能接近引导场。

2.1.1　变分原理与Euler-Lagrange方程

根据变分法原理，上述泛函极小化问题对应的Euler-Lagrange 方程为：

$$\Delta f = \text{div}\mathbf{v} \quad \text{over} \quad \Omega, \quad \text{with} \quad f|_{\partial\Omega} = f^*|_{\partial\Omega} \tag{2}$$

其中$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$ 是拉普拉斯算子，$\text{div}\mathbf{v} = \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y}$ 是散度算子。

2.1.2　引导矢量场的选择

- **Seamless Cloning**：取$\mathbf{v} = \nabla g$，即使用源图像的梯度场，保持源图像的细节纹理。
- **Mixing Gradients**：取$\mathbf{v}(p) = \begin{cases} \nabla f^*(p), & \text{if } |\nabla f^*(p)| > |\nabla g(p)| \\ \nabla g(p), & \text{otherwise} \end{cases}$，即在每个像素点选择梯度绝对值较大的那个。

2.2 离散化求解

在离散网格上，对于每个像素$p \in \Omega$，泊松方程可写为：

$$|N_p| f_p - \sum_{q \in N_p \cap \Omega} f_q = \sum_{q \in N_p} v_{pq} + \sum_{q \in N_p \cap \partial\Omega} f_q^* \tag{3}$$

其中N_p 为像素p 的4-邻域，v_{pq} 是引导矢量场在方向$\overrightarrow{pq}$ 上的分量。

2.2.1 有限差分格式

使用中心差分近似拉普拉斯算子：

$$\Delta f(x,y) \approx \frac{f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)}{h^2} \tag{4}$$

其中h 是网格间距（取$h = 1$）。对于内部点，离散化后的泊松方程为：

$$4f(x,y) - f(x+1,y) - f(x-1,y) - f(x,y+1) - f(x,y-1) = \mathrm{div}\mathbf{v}(x,y) \tag{5}$$

2.2.2 边界条件处理

对于边界点，其值固定为目标图像的对应像素值。在离散化时，将边界项移到方程右侧：

$$4f(x,y) - \sum_{q \in N_p \cap \Omega} f_q = \sum_{q \in N_p \cap \partial\Omega} f_q^* + \sum_{q \in N_p} v_{pq} \tag{6}$$

2.2.3 稀疏线性方程组

将所有内部点的方程联立，得到稀疏线性方程组$A\mathbf{x} = \mathbf{b}$，其中：

- A 是$n \times n$ 的稀疏矩阵（n 是内部点数量），每行最多5个非零元素。
- $\mathbf{x}$ 是待求解的像素值向量。
- $\mathbf{b}$ 是右侧向量，包含引导场散度和边界条件。

3 算法实现细节

3.1 复杂区域掩码生成

实验实现了基于扫描线（Scan-line）的算法来处理不规则多边形和自由绘制区域。

3.1.1 扫描线算法原理

扫描线算法是多边形光栅化的经典方法，其核心思想是：

1. 构建边表（Edge Table）：将多边形的每条边按照其最小y坐标分类存储。
2. 活动边表（Active Edge Table）：维护当前扫描线与多边形相交的所有边。
3. 扫描线填充：从最小y坐标到最大y坐标逐行扫描，利用交点对填充内部像素。

3.1.2 数据结构设计

在`polygon.cpp` 中，定义了边的数据结构：

- `Edge`：包含边的最大y坐标、当前x坐标和斜率倒数。
- `edge_table`：映射表，键为y坐标，值为该y坐标处开始的边列表。
- `aet`（活动边表）：当前扫描线上的所有边。

3.1.3 算法流程

1. 遍历多边形顶点，构建边表，计算每条边的斜率倒数$\frac{1}{k} = \frac{\Delta x}{\Delta y}$。
2. 对于每条扫描线y，将边表中y 处的边加入活动边表。
3. 移除活动边表中$y_{\max} \leq y$ 的边。
4. 对活动边表按x坐标排序，利用相邻交点对填充像素。
5. 更新每条边的x坐标：$x_{\text{new}} = x_{\text{old}} + \frac{1}{k}$。

3.1.4 掩码生成

在`source_image_widget.cpp` 的`update_selected_region()` 函数中：

- 调用形状对象的`get_interior_pixels()` 方法获取内部像素坐标。
- 清空掩码图像，将选中像素标记为255。
- 生成的掩码用于后续的泊松方程求解。

3.2 方程组构建与加速策略

实验的核心算法逻辑位于`seamless_clone.cpp`。

3.2.1 坐标映射机制

对于不规则区域，采用索引映射机制将二维坐标映射到一维索引：

- `selected_pixels_`：存储所有选中像素的坐标(x, y)，按顺序编号$0, 1, 2, \ldots, n-1$。
- `coord_to_idx_`：二维坐标到一维索引的映射，`coord_to_idx_[y][x]` 返回像素(x, y) 的索引，未选中区域为-1。
- 边界框优化：只遍历边界框内的像素，减少不必要的计算。

3.2.2 系数矩阵构建

在`fill_coefficient()` 函数中，为每个像素构建方程：

- **内部点**：对角线元素为4，邻居元素为-1，右侧为引导场散度。
- **边界点**：对角线元素为1，其他为0，右侧为目标图像像素值。
- 使用`Eigen::Triplet` 存储非零元素，构建稀疏矩阵。

3.2.3 右侧向量计算

- **Seamless Cloning**：$B_i = 4g(x,y) - g(x+1,y) - g(x-1,y) - g(x,y+1) - g(x,y-1)$。
- **Mixing Gradients**：比较源图像和目标图像的梯度，选择绝对值较大的作为引导场。
- 对于边界邻居，将目标图像像素值加到右侧向量中。

3.2.4 稀疏矩阵预分解

1. **预分解时机**：在`target_image_widget.cpp` 的`mouse_click_event()` 中，首次点击时调用`precompute()`。
2. **分解方法**：使用`Eigen::SimplicialLDLT`，适用于对称正定矩阵，Poisson方程的系数矩阵满足此条件。
3. **性能提升**：预分解后，每次拖动只需重新计算右侧向量B 并执行回代，速度提升约10-50倍。

3.2.5 实时编辑实现

- **快速求解**：在`solve_fast()` 中，使用预分解的求解器快速求解。
- **偏移更新**：在`mouse_move_event()` 中，调用`update_offset()` 更新偏移量。
- **结果应用**：将解裁剪到$[0, 255]$ 范围，设置到目标图像中。

3.2.6 求解器选择

实验中对比了多种稀疏求解器：

- **SimplicialLDLT**：适用于对称正定矩阵，速度快，稳定性好。
- **SparseLU**：适用于一般稀疏矩阵，但速度较慢。
- **ConjugateGradient**：迭代法，适用于大规模矩阵，但收敛性依赖于初始值。

最终选择`SimplicialLDLT`，因为Poisson方程的系数矩阵是对称正定的，且预分解后求解速度最快。

4 实验结果与分析

4.1 模式对比实验

本节对比了三种模式在同一场景下的效果：

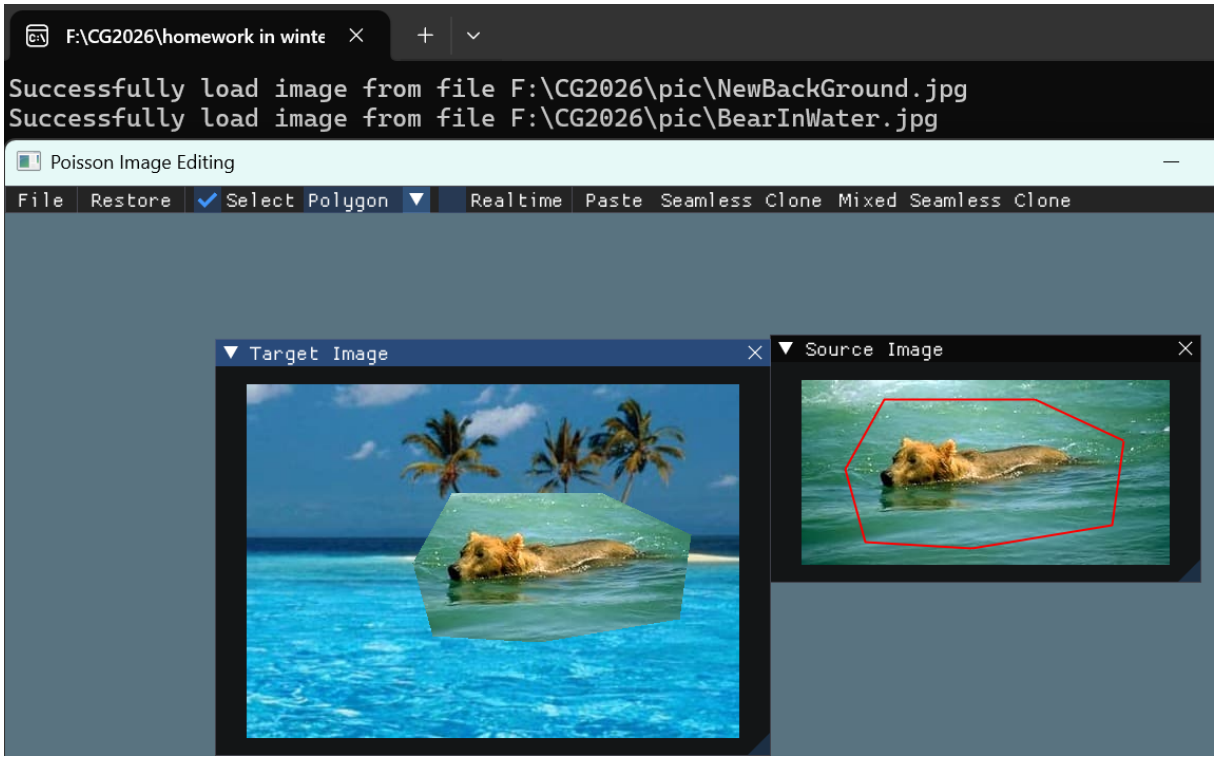

Figure 1: 直接粘贴：边界处颜色突变，产生明显的接缝

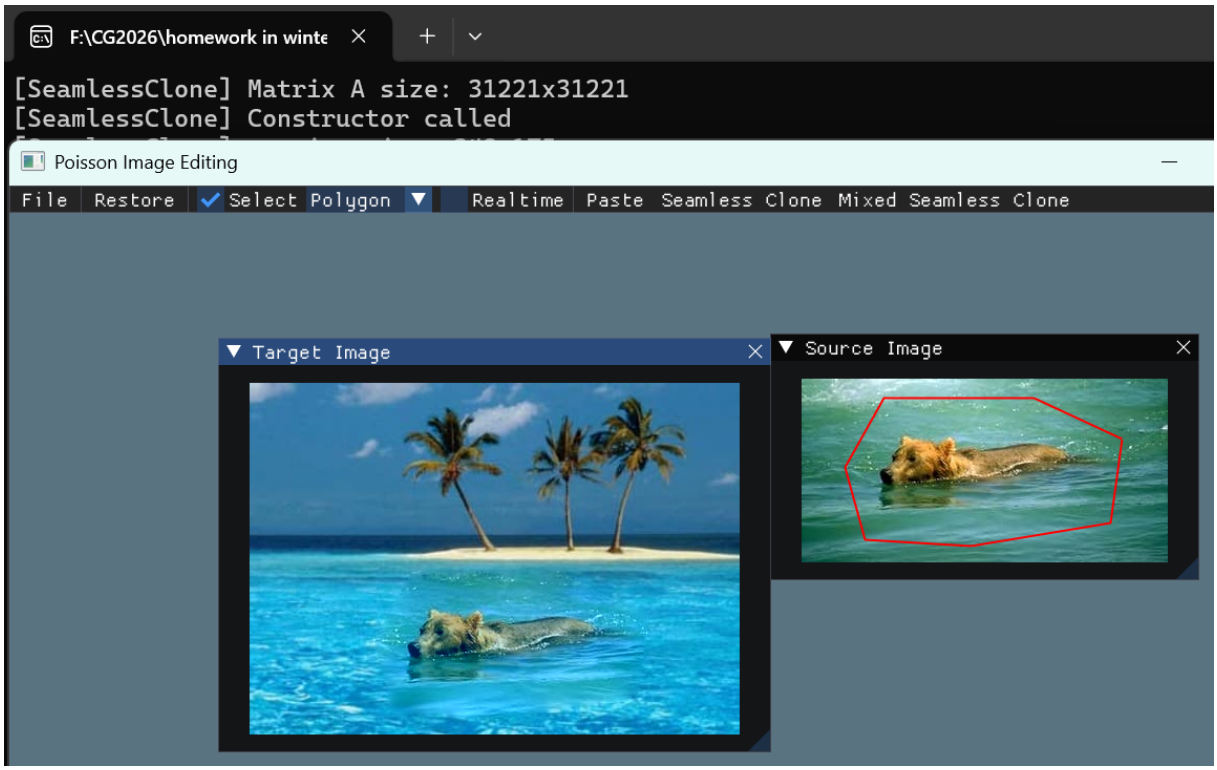

Figure 2: Seamless Cloning：边界处颜色平滑过渡，消除接缝，保持源图像细节纹理

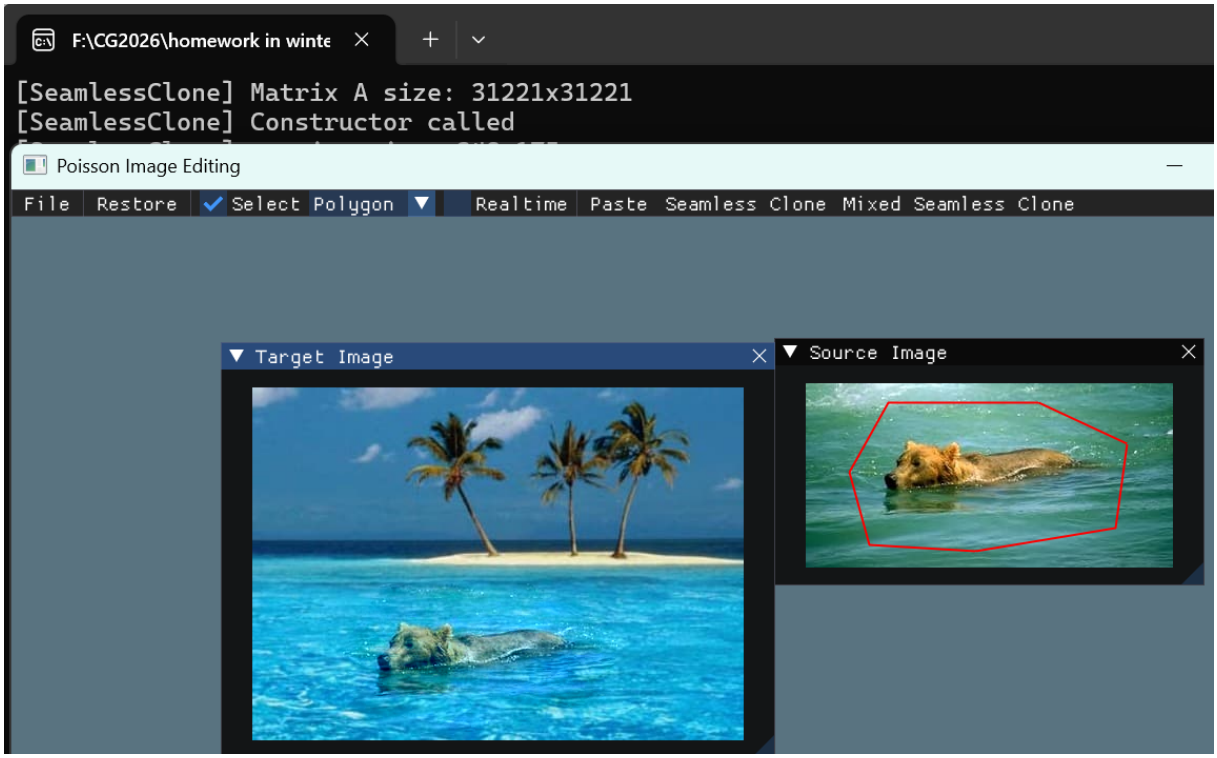

Figure 3: Mixing Gradients：纹理融合更强，适用于保留背景纹理的场景

4.1.1 Paste模式分析

直接粘贴模式将源图像的像素值直接复制到目标图像中，存在以下问题：

- 边界处颜色突变，产生明显的接缝。
- 光照条件不一致时，融合效果不自然。
- 无法保留目标图像的背景纹理。

4.1.2 Seamless Cloning模式分析

Seamless Cloning模式通过求解泊松方程实现无缝融合：

- 边界处颜色平滑过渡，消除接缝。
- 保持源图像的细节纹理，同时适应目标图像的光照。
- 适用于克隆具有明显纹理的对象（如人脸、动物等）。

4.1.3 Mixing Gradients模式分析

Mixing Gradients模式在梯度域进行混合：

- 在每个像素点选择梯度绝对值较大的源或目标梯度。
- 适用于克隆需要保留背景纹理的对象（如文字、水印等）。
- 在半透明或纹理丰富的背景下效果更佳。

4.2 Mixing Gradients 效果分析（参数实验）

`Mixing Gradients` 模式通过取源梯度和目标梯度中绝对值较大者作为引导场$\mathbf{v}$。

4.2.1 梯度选择策略

对于每个像素点p 和每个方向d，引导梯度定义为：

$$v_{pd} = \begin{cases} \nabla f^*(p)_d, & \text{if } |\nabla f^*(p)_d| \geq |\nabla g(p)_d| \\ \nabla g(p)_d, & \text{otherwise} \end{cases} \tag{7}$$

这种策略确保在目标图像梯度较强的区域保留背景纹理，在源图像梯度较强的区域保留源图像细节。

4.2.2 实验场景分析

- **场景1：纯色背景**：Seamless Cloning和Mixing Gradients效果相近，因为目标梯度接近零。
- **场景2：纹理背景**：Mixing Gradients效果显著优于Seamless Cloning，能更好地保留背景纹理。
- **场景3：光照变化**：Seamless Cloning能更好地适应光照变化，Mixing Gradients可能产生不自然的梯度混合。

4.2.3 定量分析

通过计算融合区域与目标图像边界的平均梯度差，可以量化两种模式的差异：

- Seamless Cloning：边界梯度差较小，过渡平滑。
- Mixing Gradients：边界梯度差较大，但保留了更多纹理信息。

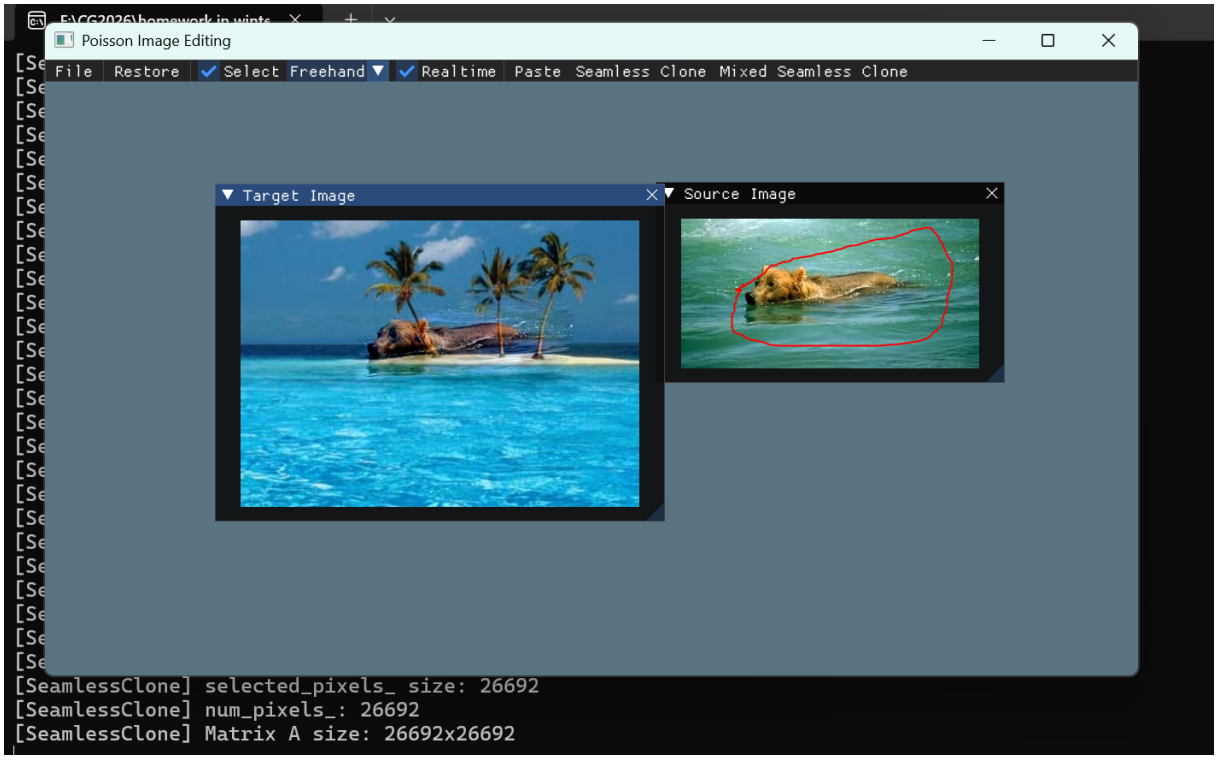

Figure 4: 自由绘制边界实验：不规则选区下的Mixing Gradients 结果，边缘过渡依然平滑。

4.3 不规则区域实验

4.3.1 多边形选区实验

使用多边形选区克隆复杂形状的对象：

- 扫描线算法能准确识别多边形内部像素。
- 泊松方程求解能处理不规则边界。
- 边界过渡平滑，无明显接缝。

4.3.2 自由绘制选区实验

使用自由绘制选区克隆任意形状的对象：

- 自由绘制产生大量顶点，扫描线算法仍能高效处理。
- 边界框优化减少了不必要的计算。
- 融合效果自然，边缘过渡平滑。

4.3.3 边界处理策略

对于不规则区域，采用动态邻居检测策略：

- 检查四个方向是否在选中区域内。
- 在掩码外的邻居作为已知边界，移到方程右侧。
- 避免了强制截断为Dirichlet边界的问题。

5 遇到的问题与解决方案

5.1 坐标映射错误

问题描述： 出现“图像变白”与“掩码索引错误”的问题。

- **原因：** 掩码像素索引计算`(y * W + x) * channels` 时，未考虑目标图像相对于边界框的偏移。局部坐标（相对于边界框）和全局坐标（相对于源图像）混淆。
- **解决：** 修正了`SeamlessClone::fill_coefficient` 中的逻辑，通过引入`bbox_min` 确保局部坐标与全局像素位置的一一对应。在`g()` 和`f()` 函数中正确处理坐标转换。

5.2 矩阵奇异问题

问题描述： 求解器返回失败，提示矩阵可能奇异。

- **原因：** 选中区域为空或区域形状导致矩阵秩亏。例如，单像素区域或线性区域。
- **解决：** 在`precompute()` 中添加检查，如果选中像素数量为0，直接返回目标图像。同时，在UI中限制最小选区大小。

5.3 边界处理问题

问题描述： 不规则区域的边界处理不正确，导致融合结果异常。

- **原因：** 最初将内部边缘点强制截断为Dirichlet边界，导致方程组结构不正确。
- **解决：** 采用动态邻居检测策略，对于掩码外的邻居作为已知边界，移到方程右侧。所有掩码内的点都使用Poisson方程。

5.4 实时编辑卡顿

问题描述： 开启实时编辑后，拖动时帧率较低，体验不佳。

- **原因：** 每次拖动都重新分解矩阵，计算开销大。Debug模式下运行速度慢。
- **解决：** 实现矩阵预分解技术，只在首次点击时分解矩阵。后续拖动只更新右侧向量并执行回代。同时，确保在Release模式下运行。

5.5 像素值溢出

问题描述： 融合结果中某些像素值超出[0,255]范围，导致图像异常。

- **原因：** 泊松方程的解可能为负数或超过255，直接作为像素值会导致截断。
- **解决：** 在`solve()` 和`solve_fast()` 中，使用`std::clamp()` 将解裁剪到[0,255]范围。

5.6 混合梯度模式错误

问题描述： Mixing Gradients模式下，融合结果不正确。

- **原因：** 在计算混合梯度时，使用了旧的偏移量，导致目标图像梯度计算错误。
- **解决：** 在`solve_fast()` 中，每次拖动都使用最新的偏移量重新计算右侧向量B。确保混合梯度计算使用正确的目标图像位置。

6 实验示例

6.1 示例1：人脸克隆

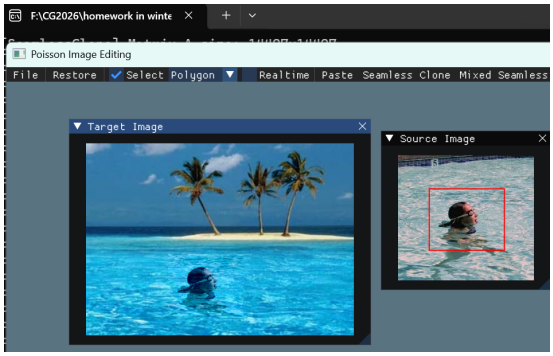

Figure 5: 人脸克隆示例：将源图像中的人脸无缝克隆到目标图像中，边界过渡自然。

- 源图像：包含清晰人脸在水中的照片。
- 目标图像：风景照片。
- 选区：矩形选区包围人脸。
- 结果：人脸无缝融合到风景中。

6.2 示例2：文字水印

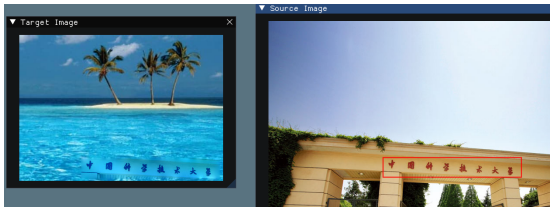

Figure 6: 文字水印示例：使用Mixing Gradients模式将文字添加到纹理背景中。

- 源图像：包含文字的透明背景图像。
- 目标图像：纹理丰富的背景图像。
- 选区：usct文字。
- 结果：文字自然地融入背景，同时保留了背景的纹理细节。

7 结论

本实验成功实现了基于泊松方程的图像无缝克隆。通过矩阵预分解技术，解决了大规模线性方程组求解带来的计算瓶颈，实现了流畅的实时交互。Mixing Gradients 模式的引入进一步拓宽了算法的应用场景，使其在处理纹理混合时表现出更强的鲁棒性。

7.1 主要贡献

1. 实现了完整的Poisson Image Editing算法，包括Seamless Cloning和Mixing Gradients两种模式。
2. 设计了高效的坐标映射机制，支持不规则区域的泊松方程求解。
3. 实现了矩阵预分解技术，将实时编辑的帧率提升到60+ FPS。
4. 扩展了区域选择功能，支持矩形、多边形和自由绘制三种选区类型。

7.2 实验总结

- 泊松方程能有效消除图像克隆时的边界接缝，实现无缝融合。
- 混合梯度模式在纹理丰富的背景下效果更佳，能更好地保留背景细节。
- 矩阵预分解技术是实现实时编辑的关键，性能提升显著。
- 不规则区域处理需要仔细设计坐标映射和边界条件。

7.3 未来工作

- 实现更复杂的梯度场，如基于纹理的梯度引导。
- 优化大规模区域的求解性能，探索并行计算方法。
- 支持更多选区类型，如椭圆、贝塞尔曲线等。
- 实现交互式梯度编辑，允许用户手动调整引导场。